Analysis Process

This page details the data collection and analysis process used by Criterion.rs. This is a bit more advanced than the user guide; it is assumed the reader is somewhat familiar with statistical concepts. In particular, the reader should know what bootstrap sampling means.

So, without further ado, let's start with a general overview. Each benchmark in Criterion.rs goes through four phases:

- Warmup - The routine is executed repeatedly to fill the CPU and OS caches and (if applicable) give the JIT time to compile the code
- Measurement - The routine is executed repeatedly and the execution times are recorded
- Analysis - The recorded samples are analyzed and distilled into meaningful statistics, which are then reported to the user
- Comparison - The performance of the current run is compared to the stored data from the last run to determine whether it has changed, and if so by how much

Warmup

The first step in the process is warmup. In this phase, the routine is executed repeatedly to give the OS, CPU and JIT time to adapt to the new workload. This helps prevent things like cold caches and JIT compilation time from throwing off the measurements later. The warmup period is controlled by the `warm_up_time` value in the Criterion struct.

The warmup period is quite simple. The routine is executed once, then twice, four times and so on until the total accumulated execution time is greater than the configured warm up time. The number of iterations that were completed during this period is recorded, along with the elapsed time.

Measurement

The measurement phase is when Criterion.rs collects the performance data that will be analyzed and used in later stages. This phase is mainly controlled by the `measurement_time` value in the Criterion struct.

The measurements are done in a number of samples (see the `sample_size` parameter). Each sample consists of one or more (typically many) iterations of the routine. The

elapsed time between the beginning and the end of the iterations, divided by the number of iterations, gives an estimate of the time taken by each iteration.

As measurement progresses, the sample iteration counts are increased. Suppose that the first sample contains 10 iterations. The second sample will contain 20, the third will contain 30 and so on. More formally, the iteration counts are calculated like so:

```
iterations = [d, 2d, 3d, ... Nd]
```

Where `N` is the total number of samples and `d` is a factor, calculated from the rough estimate of iteration time measured during the warmup period, which is used to scale the number of iterations to meet the configured measurement time. Note that `d` cannot be less than 1, and therefore the actual measurment time may exceed the configured measurement time if the iteration time is large or the configured measurement time is small.

Note that Criterion.rs does not measure each individual iteration, only the complete sample. The resulting samples are stored for use in later stages. The sample data is also written to the local disk so that it can be used in the comparison phase of future benchmark runs.

Analysis

During this phase Criterion.rs calculates useful statistics from the samples collected during the measurement phase.

Outlier Classification

The first step in analysis is outlier classification. Each sample is classified using a modified version of Tukey's Method, which will be summarized here. First, the interquartile range (IQR) is calculated from the difference between the 25th and 75th percentile. In Tukey's Method, values less than (25th percentile - 1.5 * IQR) or greater than (75th percentile + 1.5 * IQR) are considered outliers. Criterion.rs creates additional fences at (25pct - 3 * IQR) and (75pct + 3 * IQR); values outside that range are considered severe outliers.

Outlier classification is important because the analysis method used to estimate the average iteration time is sensitive to outliers. Thus, when Criterion.rs detects outliers, a warning is printed to inform the user that the benchmark may be less reliable. Additionally, a plot is generated showing which data points are considered outliers, where the fences are, etc.

Note, however, that outlier samples are *not* dropped from the data, and are used in the following analysis steps along with all other samples.

Linear Regression

The samples collected from a good benchmark should form a rough line when plotted on a chart showing the number of iterations and the time for each sample. The slope of that line gives an estimate of the time per iteration. A single estimate is difficult to interpret, however, since it contains no context. A confidence interval is generally more helpful. In order to generate a confidence interval, a large number of bootstrap samples are generated from the measured samples. A line is fitted to each of the bootstrap samples, and the result is a statistical distribution of slopes that gives a reliable confidence interval around the single estimate calculated from the measured samples.

This resampling process is repeated to generate the mean, standard deviation, median and median absolute deviation of the measured iteration times as well. All of this information is printed to the user and charts are generated. Finally, if there are saved statistics from a previous run, the two benchmark runs are compared.

Comparison

In the comparison phase, the statistics calculated from the current benchmark run are compared against those saved by the previous run to determine if the performance has changed in the meantime, and if so, by how much.

Once again, Criterion.rs generates many bootstrap samples, based on the measured samples from the two runs. The new and old bootstrap samples are compared and their T score is calculated using a T-test. The fraction of the bootstrapped T scores which are more extreme than the T score calculated by comparing the two measured samples gives the probability that the observed difference between the two sets of samples is merely by chance. Thus, if that probability is very low or zero, Criterion.rs can be confident that there is truly a difference in execution time between the two samples. In that case, the mean and median differences are bootstrapped and printed for the user, and the entire process begins again with the next benchmark.

This process can be extremely sensitive to changes, especially when combined with a small, highly deterministic benchmark routine. In these circumstances even very small changes (eg. differences in the load from background processes) can change the measurements enough that the comparison process detects an optimization or regression. Since these sorts of unpredictable fluctuations are rarely of interest while benchmarking, there is also a configurable noise threshold. Optimizations or regressions within (for example) +-1% are considered noise and ignored. It is best to benchmark on a quiet computer where possible to minimize this noise, but it is not always possible to eliminate it entirely.